

- Set up the buzzer pin

2. Main loop.

- Get the current time from the RTC
- Check the current time against preset times for the prayer or national anthem and each period bell
 - If the current time matches a preset time, play the audio or sound the buzzer
 - Use a delay to avoid busy-waiting and reduce CPU usage

Detailed code explanation

1. Initialisation.

`Serial.begin(9600)`: Initialises serial communication at a baud rate of 9600 for debugging

`SD.begin(SD_CS_PIN)`: Initialises the SD card. If it fails, a message is printed

`Wire.begin()`: Initialises I2C communication for the RTC

`rtc.begin()`: Initialises the RTC. If it fails, a message is printed

`tmrpcm.speakerPin = AUDIO_OUT_PIN`: Sets the pin for audio output

`tmrpcm.setVolume(5)`: Sets the volume level

`pinMode(buzzer_pin, output)`: Sets the buzzer pin as an output

2. Main loop explanation.

`rtc.now()`: Gets the current time from the RTC

`Serial.print()`: Prints the current time to the serial monitor for debugging

`digitalWrite(buzzer_pin, low)`: Ensures the buzzer is off initially

Time checks: Checks if the current time matches any of the preset times. If a match is found

`playAudio()`: Plays the prayer or national anthem, if it is the starting time

`digitalWrite(buzzer_pin, high)`: Turns on the buzzer

`delay(30000)`: Keeps the buzzer on for 30 seconds.

`digitalWrite(buzzer_pin, low)`: Turns off the buzzer

`delay(1000)`: Adds a delay of one second

Circuit and working

Fig. 3 shows the circuit diagram of the automation system. It consists of Arduino Uno, real-time clock, micro-

card SD module, and 3.5mm audio female connector. The Arduino program is designed to automate the ringing of bells and playing of audio files according to a predefined schedule. It utilises various libraries, including `Wire.h` for I2C communication, `SD.h` for SD card interfacing, `TMRpcm.h` for audio playback, and `RTCLib.h` for real-time clock functionality.

The setup function initialises serial communication, SD card, RTC module, TMRpcm library, and sets the volume level. It also configures the buzzer pin as an output. In the loop function, it continuously reads the current time from the RTC module and checks if it matches any of the predefined time intervals for bell ringing.

If a match is found, it triggers the buzzer, plays the audio file (if available), and then waits for a specified duration before turning off the buzzer.

Microcard SD module. The pins on an SD card module serve the following functions:

1. VCC: This pin provides power to the SD card module. It is usually connected to a 5V or 3.3V power source, depending on the module's specifications

2. GND: This pin is the ground connection, providing the reference voltage for the module

3. MISO (master in slave out): This pin is used for data transfer from the SD card to the microcontroller or other device. It carries data from the SD card to the microcontroller when reading data

4. MOSI (master out slave in): This pin is used for data transfer from the microcontroller to

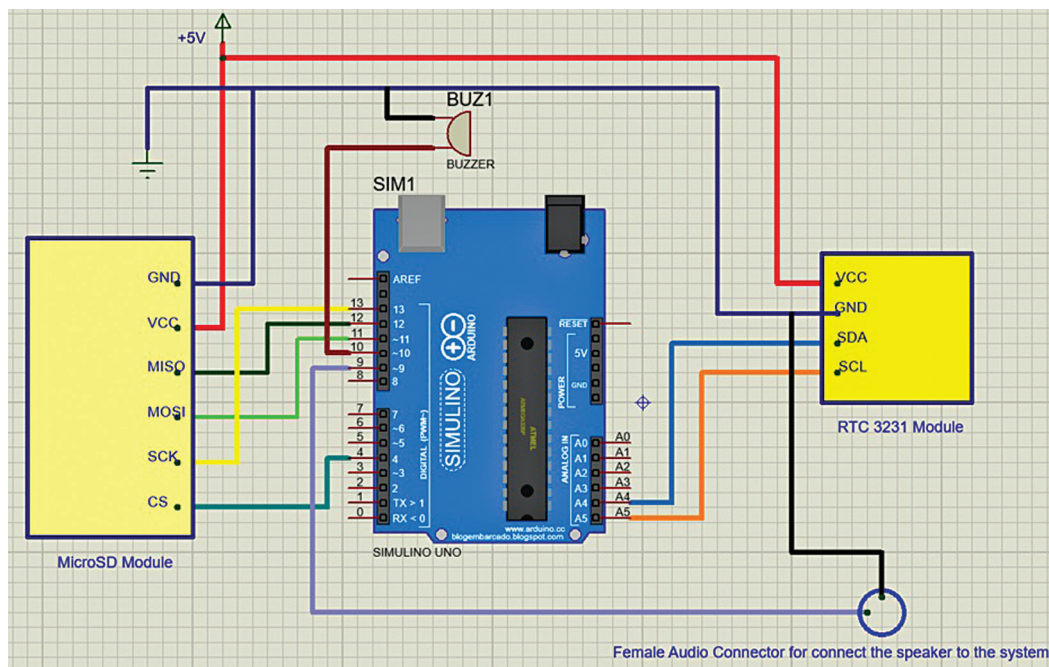


Fig. 3: Circuit diagram